

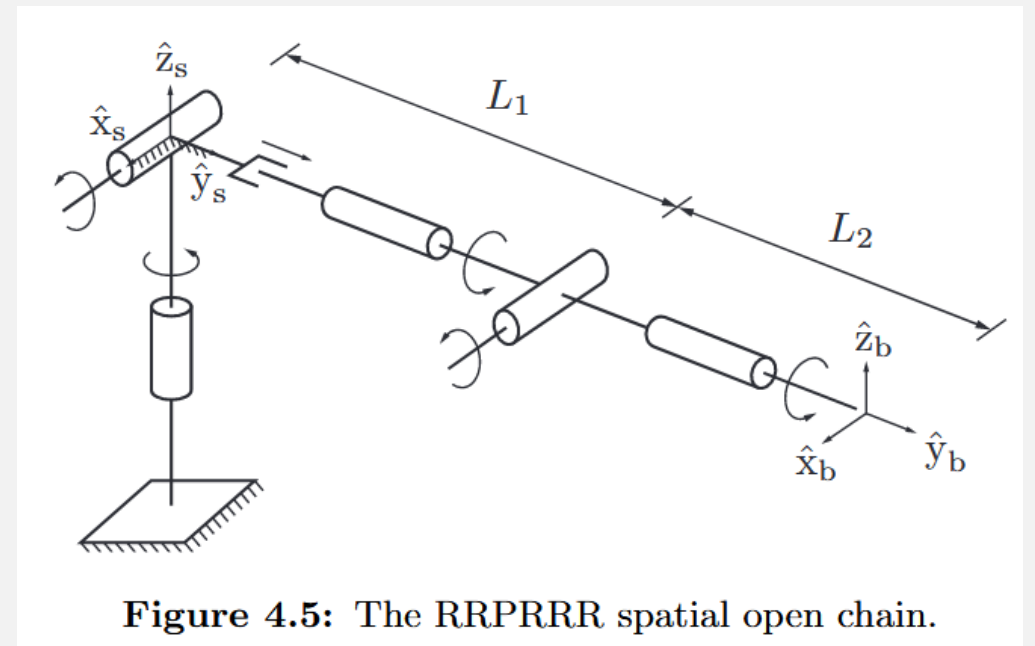
AULA 6

Sensores, odometria, e visualização no RViz

EXERCÍCIO DA ÚLTIMA AULA

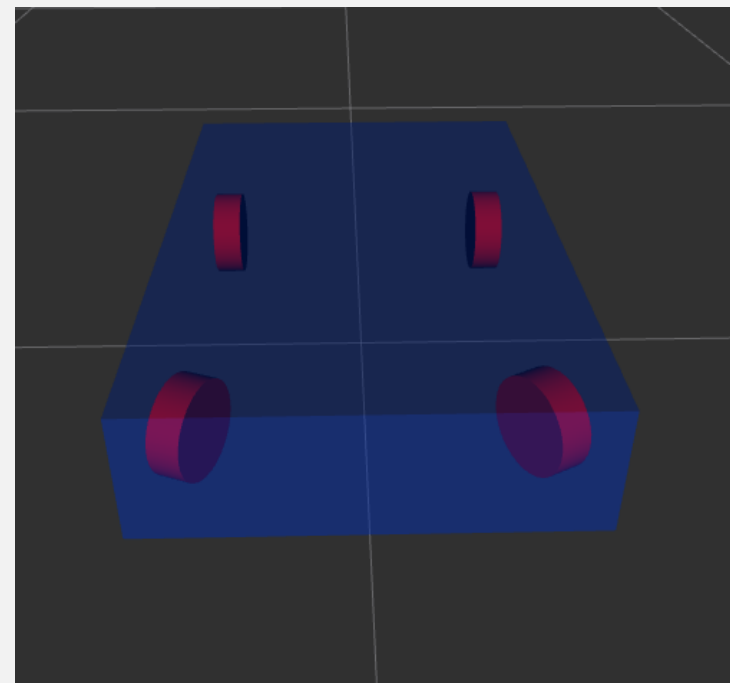
Faça, em URDF, o seguinte modelo de robô:

- Definição dos frames é de livre escolha
- Faça um pacote para fazer o launch do mesmo no RViz, junto com os nós `robot_state_publisher` e `joint_state_publisher_gui`
- Recomendado: use a extensão `xacro` para modularizar o seu código, fazer cálculos de inércia, etc...



EXERCÍCIO DA ÚLTIMA AULA 2

Altere o URDF do carro da última aula para que as rodas dianteiras rodem juntas, sendo controladas apenas por um joint



NAVEGAÇÃO E ODOMETRIA

- Navegação é uma das tarefas mais difíceis para um robô móvel fazer. 4 subtarefas:
 - Percepção (extrair dados úteis dos sensores)
 - Localização (determinar a posição do robô no ambiente)
 - Cognição (decidir como agir para atingir seus objetivos)
 - Controle de movimento (comandar atuadores para seguir trajetória desejada)

PERCEPÇÃO COMPUTACIONAL EM ROBÓTICA

- Robôs precisam ser capazes de “sentir” a si mesmos (propriocepção) e o mundo ao seu redor (exterocepção)
- Entradas do processo de percepção vêm em duas formas:
 - 1) Dados digitais obtidos de sensores/transdutores
 - 2) Modelo (parcial) que contém informação sobre estado do robô e entidades relevantes
- Sensores obtêm dados (1) de “baixo nível” (ex: ângulo de juntas, imagens de câmeras, distância de um laser)
- Estima-se o estado do robô a partir desses dados (1) em conjunção com o modelo do robô/mundo (2)
 - posição, orientação, velocidade
- Em um nível mais alto, essa informação é usada para o robô tomar decisões e planejar o que fazer

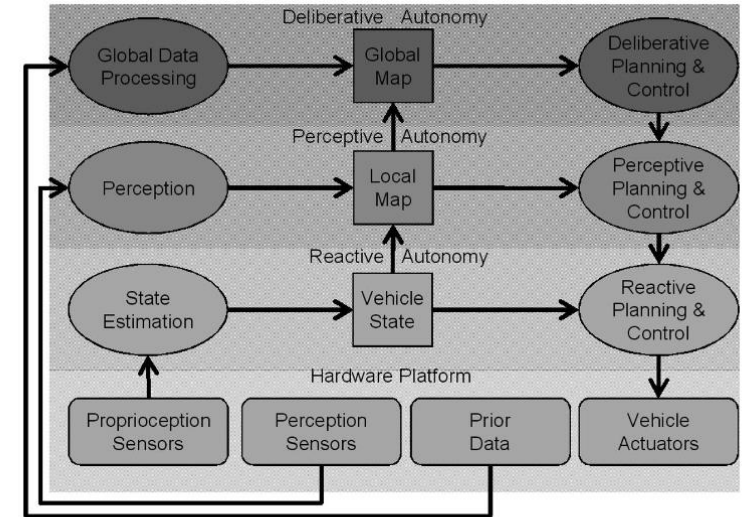
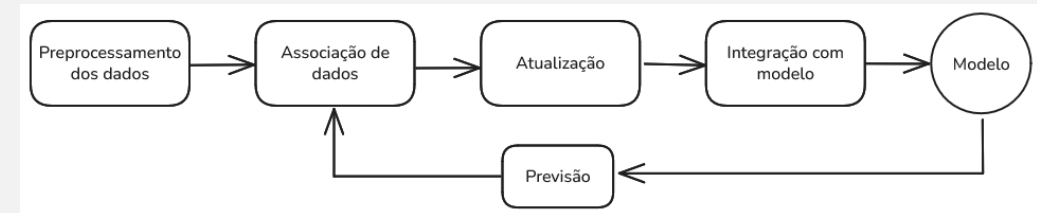
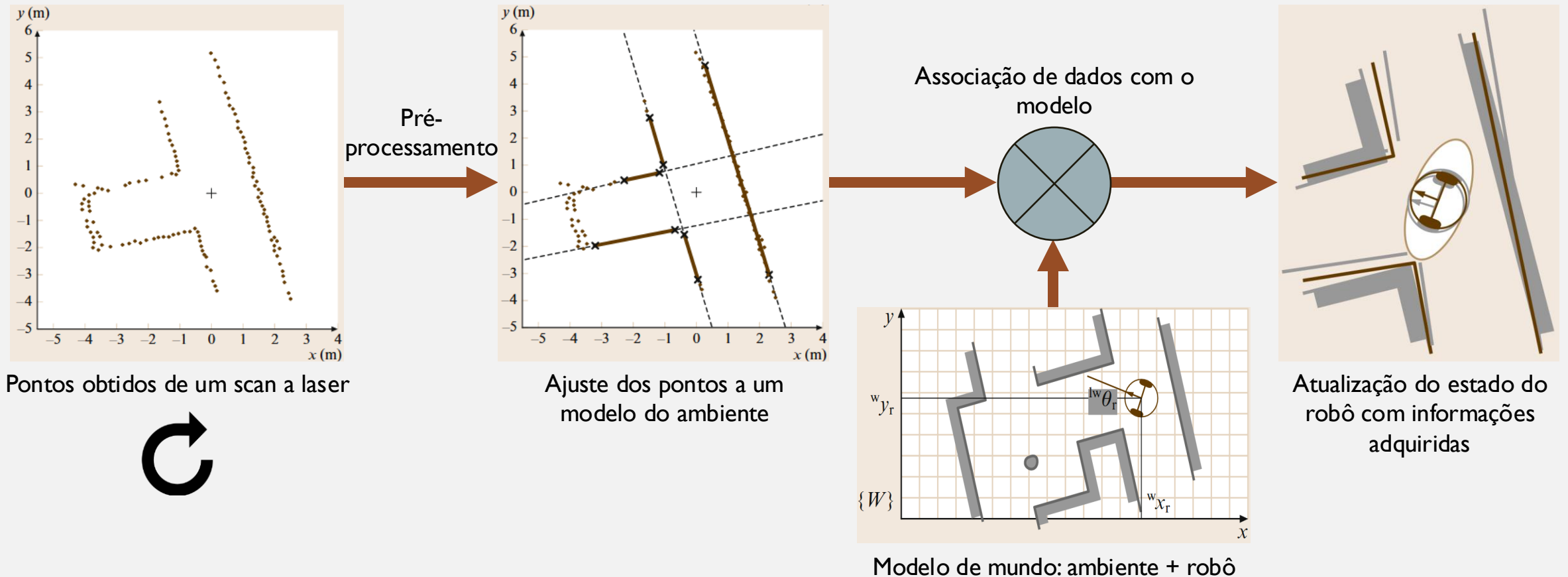


Figure 1.9 Layers of Autonomy. The entire mobile system can be described in terms of three nested perceive-think-act loops.

Camadas de autonomia de robôs móveis

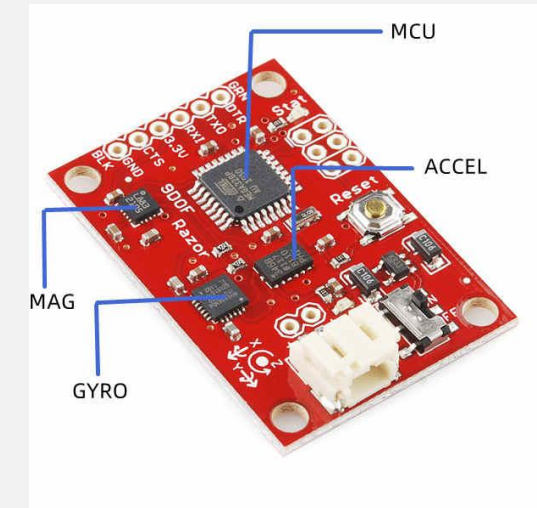


PERCEPÇÃO COMPUTACIONAL EM ROBÓTICA



PERCEPÇÃO COMPUTACIONAL EM ROBÓTICA

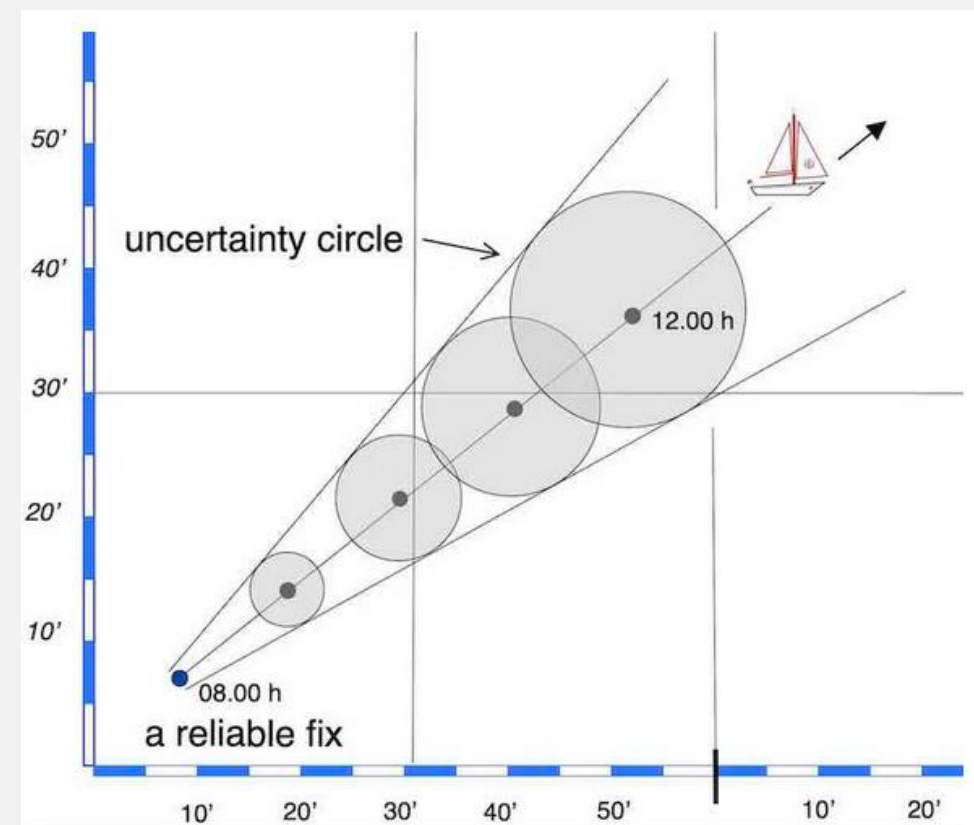
- Sensores podem ser divididos em incrementais ou absolutos
 - Incrementais: mudança do estado atual com relação ao passado
 - Processamento simples, alta velocidade, sujeitos a *drift*
 - Ex: acelerômetros, giroscópios, encoders incrementais, fluxo ótico
 - Absolutos: estado atual é mensurado independente do histórico
 - Imunes a *drift*, mas mais lentos e requerem processamento sofisticado
 - Ex: GPS, magnetômetro, sonar, laser, *landmarks* ou mapas
- Uma plataforma robótica usualmente contém vários de cada
 - **Fusão de sensores:** podemos combinar informações desses vários sensores para diminuir a incerteza sobre o estado do robô!
 - i.g.: Filtro de Kalman, Filtro de partículas, Lógica fuzzy, ...



IMUs combinam acelerômetros e giroscópios de 3 eixos (sensores incrementais), e por vezes também um magnetômetro (sensor absoluto), em um pacote só

ODOMETRIA/DEAD-RECKONING

- Uso de dados de sensores proprioceptivos e modelo cinemático para estimar a mudança de posição ao longo do tempo
 - Basicamente : dado que nosso robô se move de tal maneira, se ele tem essa velocidade e aceleração onde estará após um tempo
- Altamente sujeito a erro sistemático de deriva
- Robôs terrestres usam o movimento dos atuadores (rodas, lagartas, pernas)
 - Modelos cinemáticos tipicamente não levam em conta atrito/deslize com o chão
- Dados de localização absoluta são essenciais para evitar a deriva ao longo do tempo



Estimação por modelo cinemáticos é usada há muitos anos, desde as navegações

ODOMETRIA/DEAD-RECKONING

- Ex: robôs com acionamento diferencial de rodas
- Modelo cinemático: posição $q = [x_W(k) \quad y_W(k) \quad \theta_W(k)]^T$
- Velocidade:

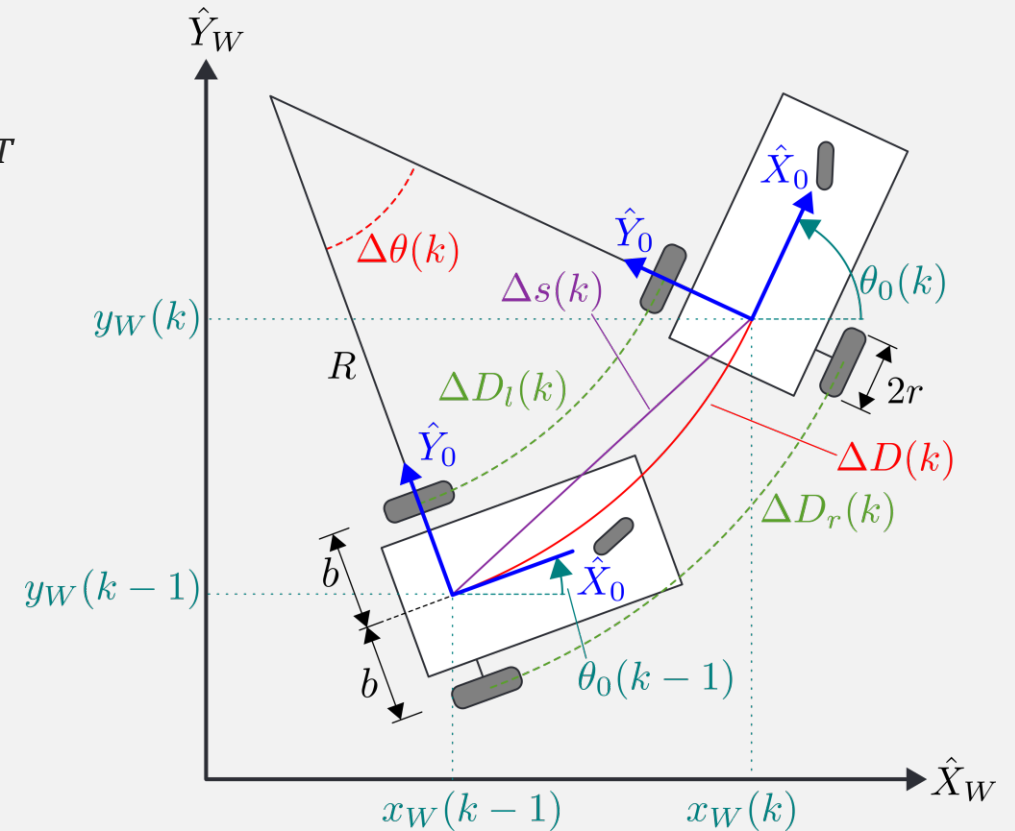
$$\dot{q} = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} v \cos \theta \\ v \sin \theta \\ \omega \end{bmatrix}, \text{ onde } v = \frac{r}{2} (\dot{\psi}_r + \dot{\psi}_l), \omega = \frac{r}{2b} (\dot{\psi}_r - \dot{\psi}_l)$$

R : raio da curva, r : raio das rodas,
 b : metade da distância entre rodas,
 ψ : ângulo de giro da roda

- Discretizando e integrando no tempo, com θ constante

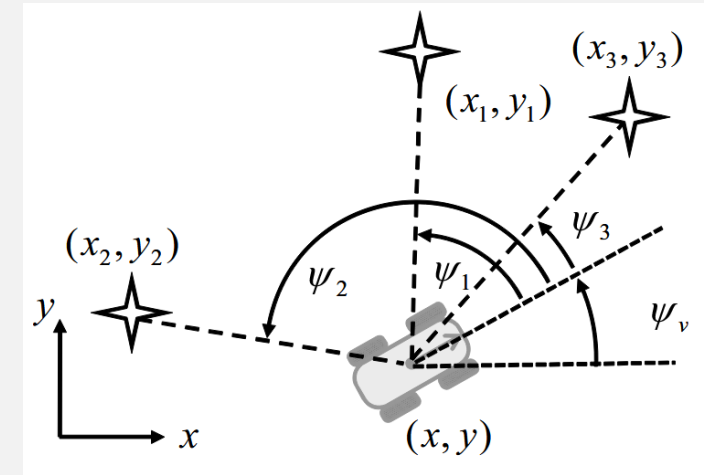
$$q_k = q_{k-1} + \dot{q}_{k-1} \Delta t = \begin{bmatrix} x_{k-1} \\ y_{k-1} \\ \theta_{k-1} \end{bmatrix} + \begin{bmatrix} v_{k-1} \cos \theta \\ v_{k-1} \sin \theta \\ \omega_{k-1} \end{bmatrix} \Delta t$$

- É possível reescrever $\dot{q}_{k-1} \Delta t$ para chegar a $\dot{q}_{k-1} \Delta t = \begin{bmatrix} \Delta s_k \cos \left(\theta_{k-1} + \frac{\Delta \theta_k}{2} \right) \\ \Delta s_k \sin \left(\theta_{k-1} + \frac{\Delta \theta_k}{2} \right) \\ \Delta \theta_k \end{bmatrix}$, onde $\Delta s_k = \Delta D_k \frac{\sin(\Delta \theta_k/2)}{\Delta \theta_k/2}$

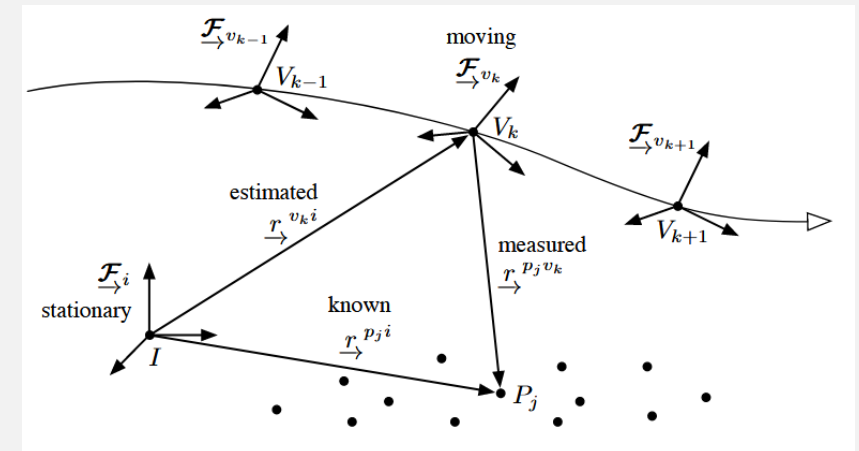


POSE-FIXING/TRIANGULAÇÃO

- Uso de medidas de distância/ângulos do robô com relação a *landmarks* para encontrar posição mais provável
- Erros não se propagam com o tempo, mas requer que o robô tenha um mapa do ambiente
- Teremos um sistema de equações não-lineares a resolver
 - Minimizamos o resíduo ponderado desse sistema
- Caso geral: dado m pontos mensurados no frame do robô ${}^R P_j$, com posição conhecida no frame inercial ${}^I P_j$, qual é a pose do robô no frame inercial, i.e. ${}^I T_R$?
 - Isso é um problema de ajuste de *point clouds* – já implementado em várias bibliotecas (e.g. [PCL](#))



Exemplo de triangulação no plano: com 3 pontos com posições conhecidas, mensurados sem ruído é possível fixar *exatamente* a posição e orientação do robô



Caso geral de triangulação instantânea – determinar transformação entre *frames* estacionário e móvel

FUSÃO DE SENSORES

- Se temos vários sensores podemos diminuir a incerteza de nossas medições
 - Mitigar drift, aumentar taxa de amostragem da estimação de estado, robustez a ruído
- Sensores incrementais e absolutos são complementares nesse aspecto
 - Defeitos de um tipicamente não ocorrem no outro
 - Sensor incrementais funcionam como predição enquanto absolutos trazem um limite de errado confiável
 - e.g.: GPS + Acelerômetros:
- A fusão pode ter diferentes características:
 - **Nível de acoplamento de dados**
 - ⑩ Baixo : Fusão com de dados já processados
 - ⑩ Alto : Fusão com os dados pré processamento
 - **Centralizada ou Distribuída**
 - Centralizado: um único estimador (filtro/otimização).
 - Distribuído: múltiplos estimadores locais trocam informações
- **Nível de processamento onde a fusão ocorre**
 - Baixo : fusão de sinais brutos (ex.: IMU+encoder no nível de velocidade/pose).
 - Médio : fusão de features (ex.: pontos 3D de LiDAR + features visuais).
 - Alto : fusão de hipóteses / decisões (ex.: combinar detectores de obstáculos).

EX: FILTRO DE KALMAN

- O Filtro de Kalman é um estimador ótimo para sistemas lineares com ruído gaussiano
- Soma a predição vinda de um modelo mais uma correção vindo de medições, ambas ponderadas por uma medida de confiança
 - Predição vinda de um modelo : Modelo cinemático
 - Medições : Sensores
- 2 passos recursivos : Predição e Atualização
 - Predição
 - Predição Estado Atual = Predição do Estado Passado + Acontecimentos Presentes

$$\hat{x}_{k|k-1} = F_k \hat{x}_{k-1|k-1} + B_k u_k$$

- Covariância Prevista = incerteza anterior através do modelo + incerteza por conta do ruído de processo

$$P_{k|k-1} = F_k P_{k-1|k-1} F_k^\top + Q_k$$

EX: FILTRO DE KALMAN

- Atualização

- Inovação (por resíduo) = medido – previsto

H_k : matriz de observação (como o estado se transforma na medição)

$$y_k = z_k - H_k \hat{x}_{k|k-1}$$

- Yk grande implica em uma "surpresa" com a medição, requer maior inovação
- Grau de confiança da inovação (Covariância) = Incerteza da predição + Incerteza do sensor

$$S_k = H_k P_{k|k-1} H_k^\top + R_k$$

- Sensores ou predições pouco confiáveis geram Sk grandes
- Peso aplicado a inovação

$$K_k = P_{k|k-1} H_k^\top S_k^{-1}$$

- Se confiamos nos sensores, mas não na predição : K grande
- Se confiamos na predição, mas não nos sensores : K pequeno
- Atualizamos o estado e a confiança proporcionalmente ao erro (yk) e K

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_k y_k \quad P_{k|k} = (I - K_k H_k) P_{k|k-1}$$

SENSORES NO ROS

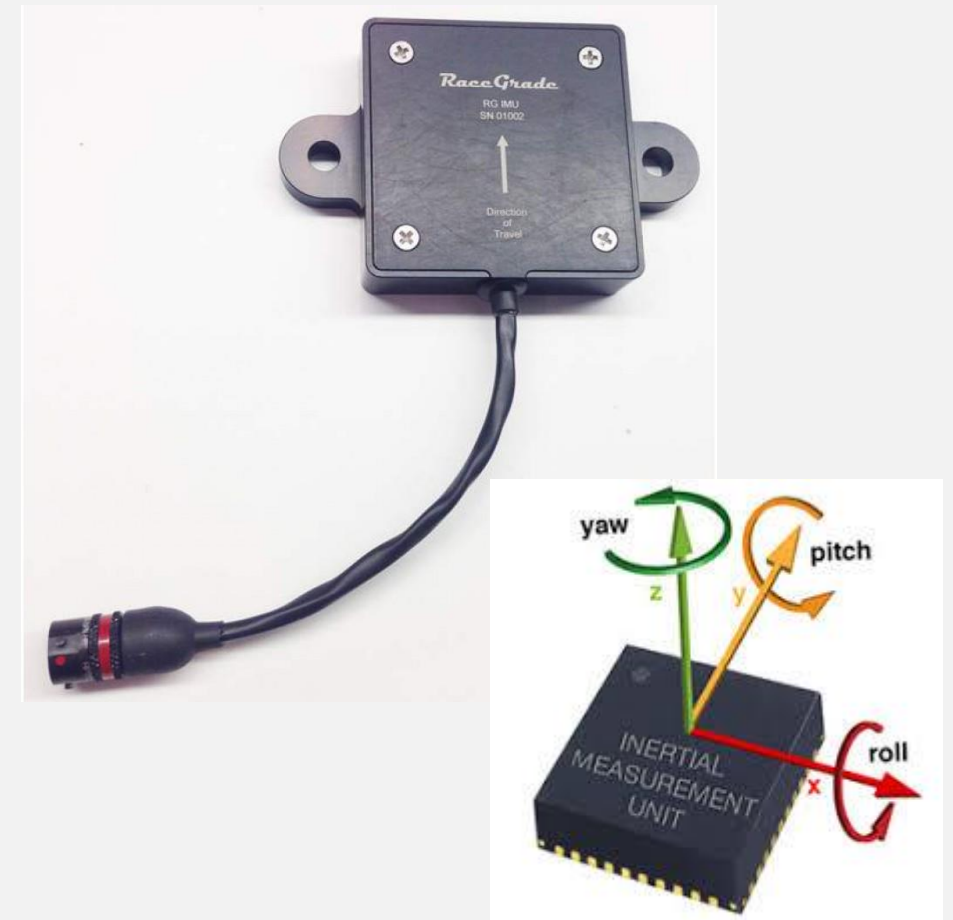
- Dados de sensores no ROS podem vir de sensores reais ou simulados
 - Em muitos casos é interessante ter uma versão *mock* do sensor a ser testado
- Tipicamente dados são publicados em tópicos com mensagens do pacote `sensor_msgs`
- Dados podem ser então usados para pós-processamento, estimação, etc...
- Vamos ver alguns dos tipos de sensores mais comuns...

sensor_msgs/msg/...	
TimeReference	Illuminance
Imu	PointCloud2
MagneticField	PointField
BatteryState	PointCloud
JointState	RegionOfInterest
MultiDOFJointState	CameraInfo
Joy	CompressedImage
JoyFeedback	Image
JoyFeedbackArray	NavSatFix
LaserEcho	NavSatStatus
LaserScan	ChannelFloat32
MultiEchoLaserScan	RelativeHumidity
Temperature	Range
FluidPressure	

SENSORES NO ROS: IMUS

Inertial Measurement Unit (Unidade de Medida Inercial)

- 3 (eventualmente mais) sensores: Acelerômetro, Giroscópio, Magnetômetro
- Sensores incrementais combinados com um absoluto
- Geralmente com altas taxas de amostragem
- Medições referentes ao próprio frame
- Muito populares para odometria ou para estimar rotações
 - Modelos mais completos até já fazem a fusão de sensores internamente



SENSORES NO ROS: IMUS

Inertial Measurement Unit (Unidade de Medida Inercial)



SENSORES NO ROS: IMUS

sensor_msgs/msg/Imu:

- std_msgs/Header header
- geometry_msgs/Quaternion orientation
 - Orientação estimada pela IMU (se disponível)
- float64[9] orientation_covariance
 - Estimativa da covariância da rotação (3 componentes em torno de cada eixo)
- geometry_msgs/Vector3 angular_velocity e float64[9] angular_velocity_covariance
 - Velocidade angular medida pelo giroscópio (rad/s) e sua covariância
- geometry_msgs/Vector3 linear_acceleration e float64[9] linear_acceleration_covariance
 - Aceleração linear medida pelo acelerômetro (m/s^2) e covariância

sensor_msgs/msg/MagneticField:

- std_msgs/Header header
- geometry_msgs/Vector3 magnetic_field e float64[9] magnetic_field_covariance
 - Componentes x,y,z do campo magnético em Teslas



Note que nem todos os IMUs contêm magnetômetros, ou estimam a orientação diretamente, ou têm informações sobre todos os eixos; os que faltam devem ser preenchidos como NaN, 0 ou -1 (ver quais exatamente com `ros2 interface show`)

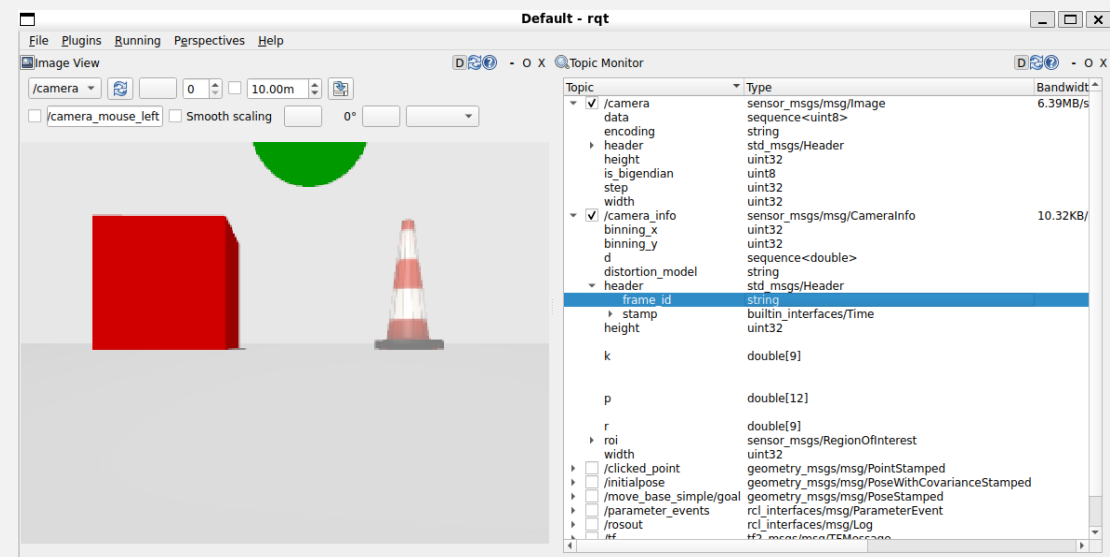
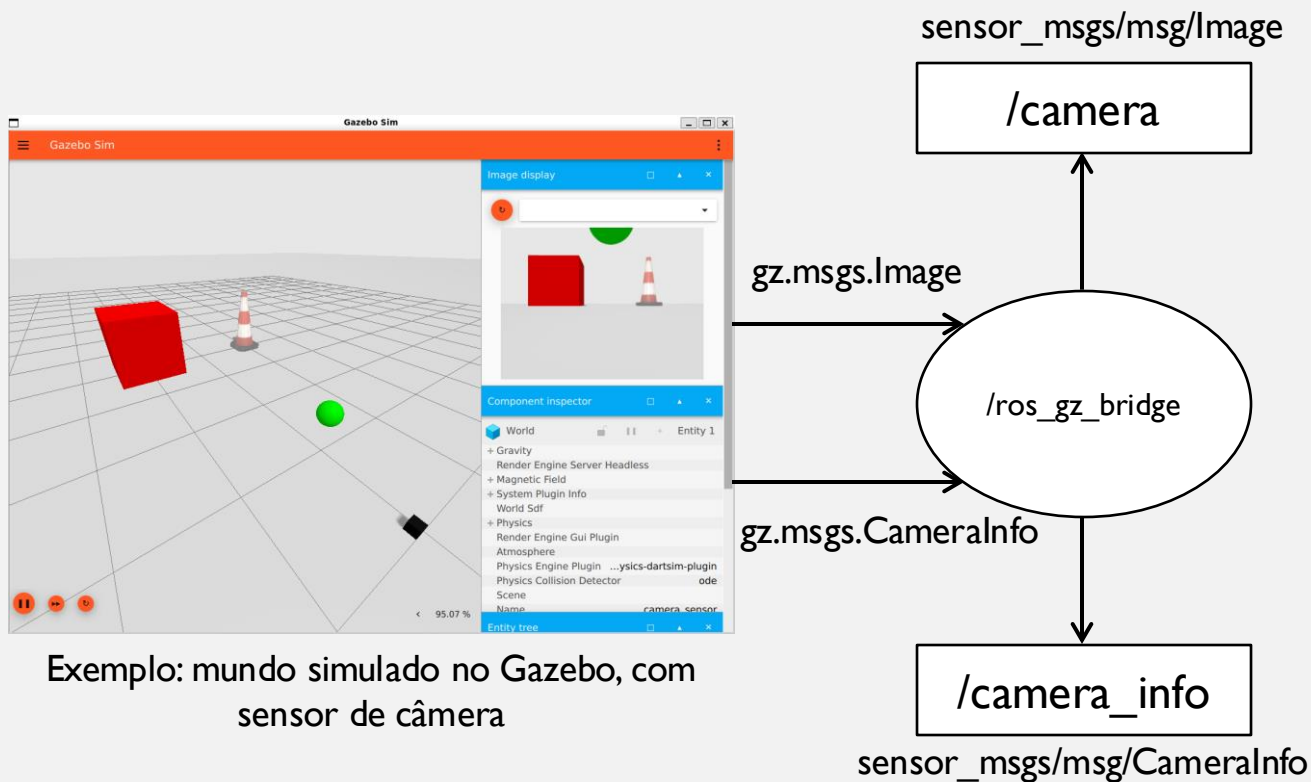
SENSORES NO ROS: CÂMERAS

Câmeras digitais:

- Monoculares, par Stereo, RGBD (informação de profundidade)
- Processamento em si é feito em algum nó para extrair informação útil
- Fabricantes usualmente disponibilizam pacotes para as suas câmeras
- Extrair informações úteis de imagens é todo o campo de visão computacional, dificilmente se têm soluções “plug-and-play”
 - Biblioteca mais popular com ROS2 é opencv
- Nativo do ROS2: mensagens `sensor_msgs` (`CameraInfo`, `Image`, `ImageCompressed`, `RegionOfInterest`)
 - Eventualmente pode ser interessante usar outros tipos
- Pacotes do grupo ROS Perception: [vision_msgs](#), [image_pipeline](#), [vision_opencv](#)

SENSORES NO ROS: CÂMERAS

Para rodar esse exemplo:
`ros2 launch ros_gz_sim_demos camera.launch.py`



Para fazer a integração entre Gazebo e Ros, usamos o metapacote [ros_gz_bridge](#).
Manualmente podemos usar o executável `parameter_bridge`, com a seguinte sintaxe:
`ros2 run ros_gz_bridge parameter_bridge /GAZEBO_TOPIC@ROS_MSG@GAZEBO_MSG`

Mais informações da integração ROS2 com o Gazebo no [site do simulador](#).

SENSORES NO ROS

Laser Rangefinder (LIDAR):

- Sensor time-of-flight
 - Transmissor ilumina o alvo com um laser, receptor detecta o componente de luz refletida
 - Tempo entre mandar e receber luz refletida é usado para estimar a distância
 - Frequentemente mede-se a defasagem da luz refletida, ao invés do tempo (construção mais barata)
- No ROS: mensagens LaserScan
- Pacotes do grupo ROS Perception: [laser_filters](#)
[laser_geometry](#) [calibration](#)

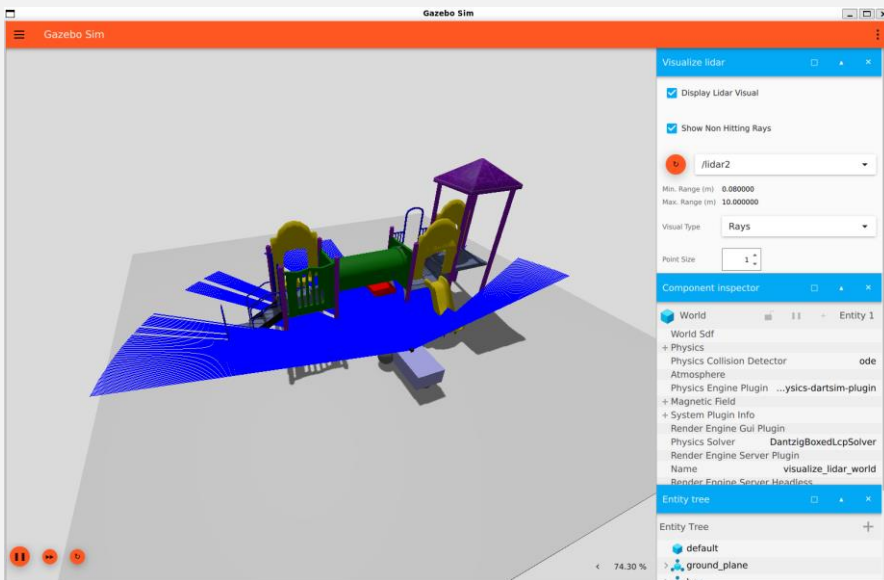


Sensor RPLiDAR CI da SLAMTEC.
Link: [Pacote para ROS 2 do fabricante](#)
O recente barateamento de LIDARs ajudou a popularizá-los em aplicações.

SENSORES NO ROS

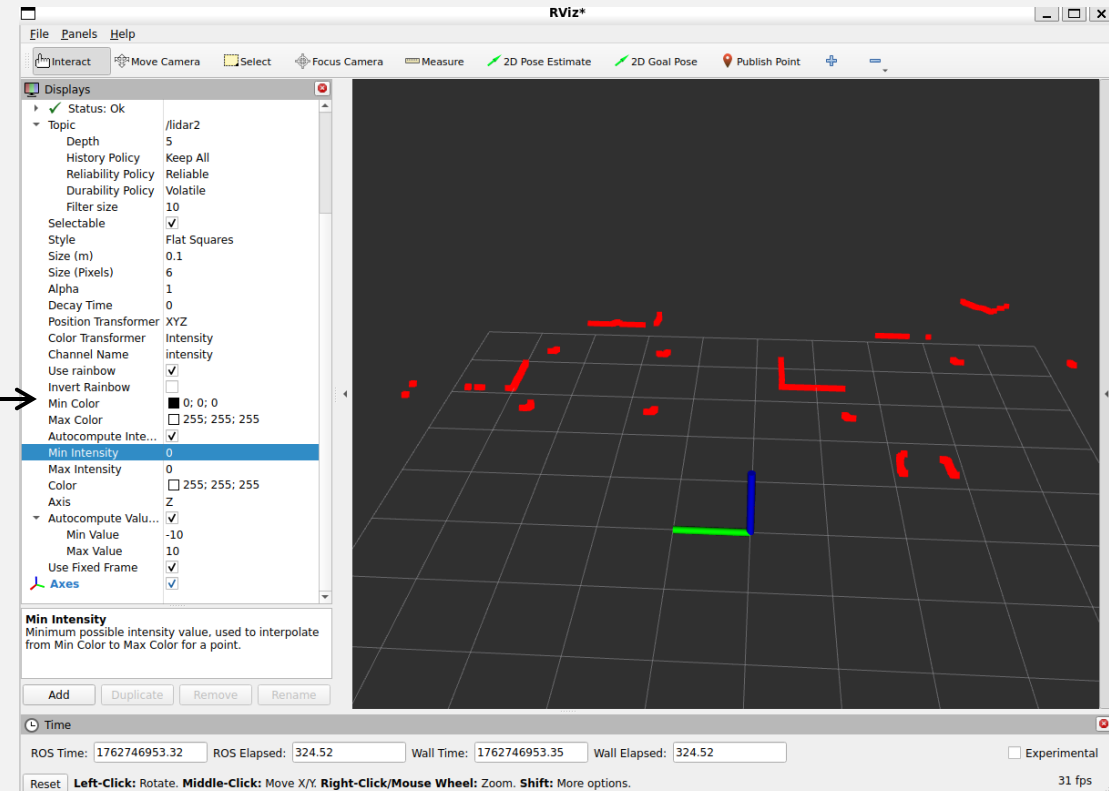
Laser Rangefinder (LIDAR):

- Exemplo: simulado no Gazebo



gz.msgs.LaserScan

sensor_msgs/msg/
LaserScan



Para rodar esse exemplo, rode em um terminal:

```
gz sim visualize_lidar.sdf &
```

```
ros2 run ros_gz_bridge parameter_bridge /lidar@sensor_msgs/msg/LaserScan@gz.msgs.LaserScan &
```

```
ros2 run tf2_ros static_transform_Publisher --frame-id map --child-frame-id model_with_lidar/link/gpu_lidar
```

Em outro terminal, lance o RViz2, selecione o display de mensagem do tipo LaserScan, e selecione o tópico /lidar

SENSORES NO ROS: LASERSCAN

`sensor_msgs/msg/LaserScan`:

- `std_msgs/Header header`
- `float32 angle_{min,max,increment}`
 - Ângulos inicial, final e intervalo entre medidas (resolução)
- `float32 time_increment`
 - Intervalo de tempo entre cada leitura
- `float32 scan_time`
 - Tempo entre scans
- `float32 range_{min,max}`
 - Valores mínimos e máximos do range
- `float32[] ranges`
 - Distância medida do LIDAR até o ponto
- `float32[] intensities`
 - Intensidade (Luminosidade) do raio refletido – mas é específico a cada dispositivo, cada fabricante implementa diferentemente

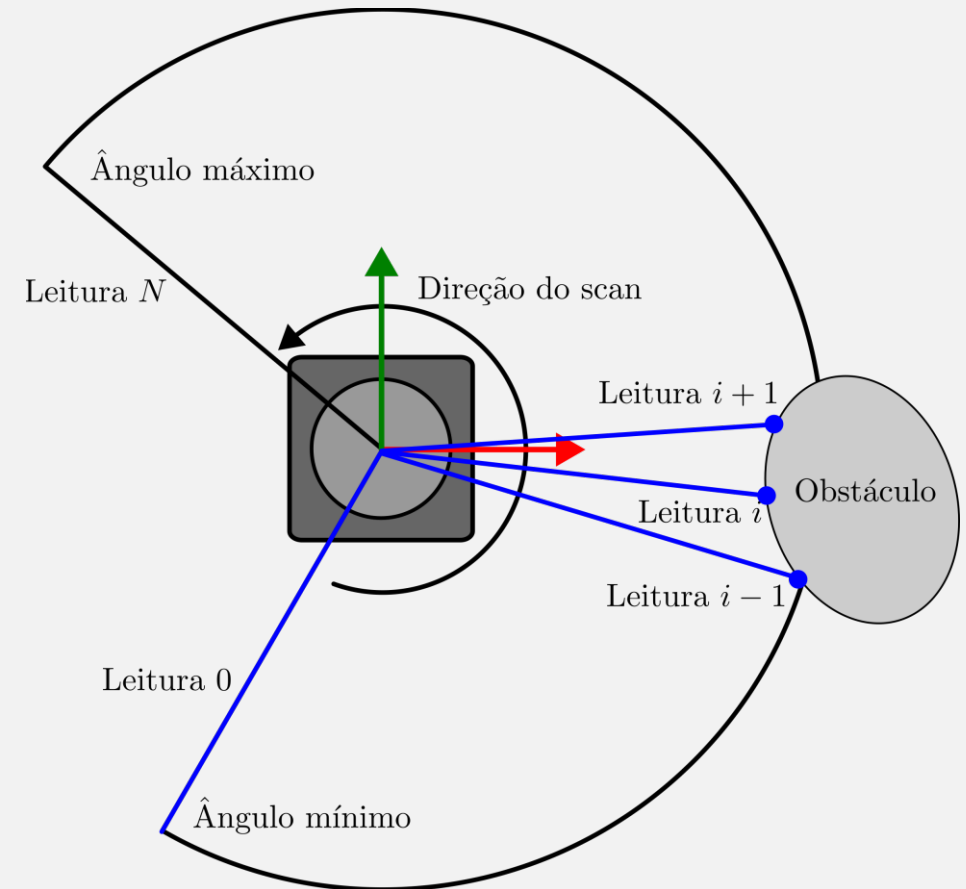
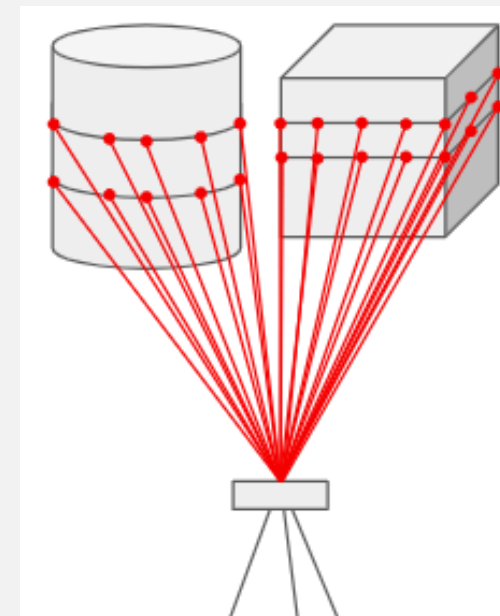


Diagrama – processo de medição de um LIDAR
(uma camada)

SENSORES NO ROS: POINTCLOUD2

`sensor_msgs/msg/PointCloud2`:

- `std_msgs/Header header`
- `uint32 height, width`
 - Estrutura 2D da PCL; se desordenada, `height=1`, `width` vira o n° de pontos.
 - Pontos vêm armazenados em um array, de forma sequencial
- `PointField[] Fields`
 - Diz como os dados estão arranjados e o que cada elemento no array significa.
 - Cada `Fields` indica um atributo (`x,y,z,intensidade, rgb`), `offset`, tipo numérico e contagem de elementos.
- `bool is_bigendian`
 - Byte mais significativo fica no fim ou no início
- `uint32 scan_time`
 - Quantos bytes ocupa um ponto
- `uint32 row_step`: Comprimento de uma linha do array em bytes
- `bool is_dense`: `true` se não tiverem pontos inválidos
- `uint8[] data`: Dados dos pontos.

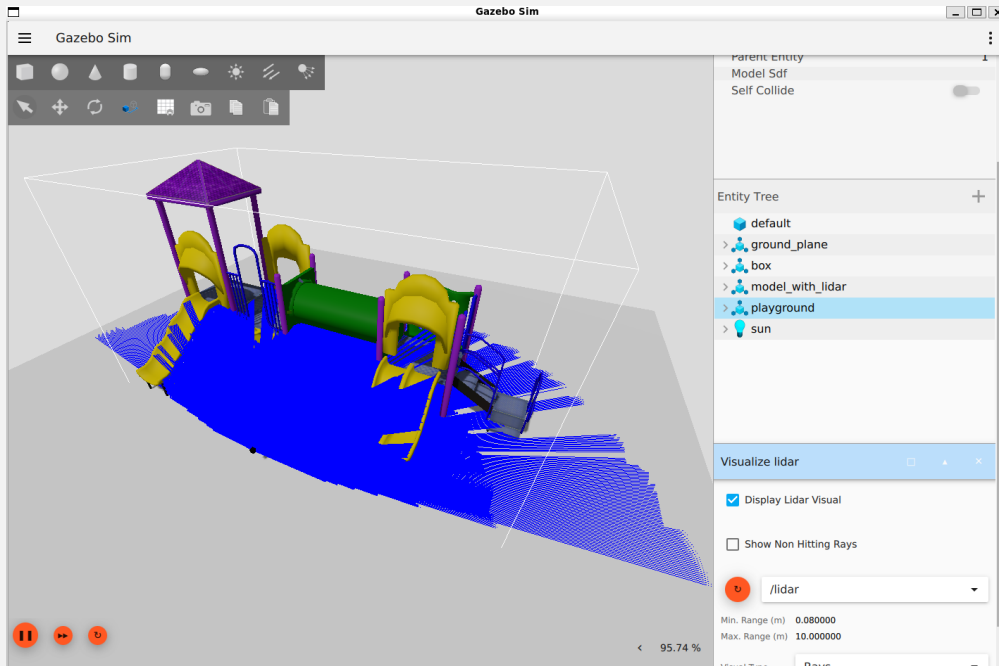


Uma point cloud pode ser gerada de várias formas
– tipicamente via LIDARs, mas também via visão computacional (par de câmeras estéreo), SONARs e outros.

SENSORES NO ROS

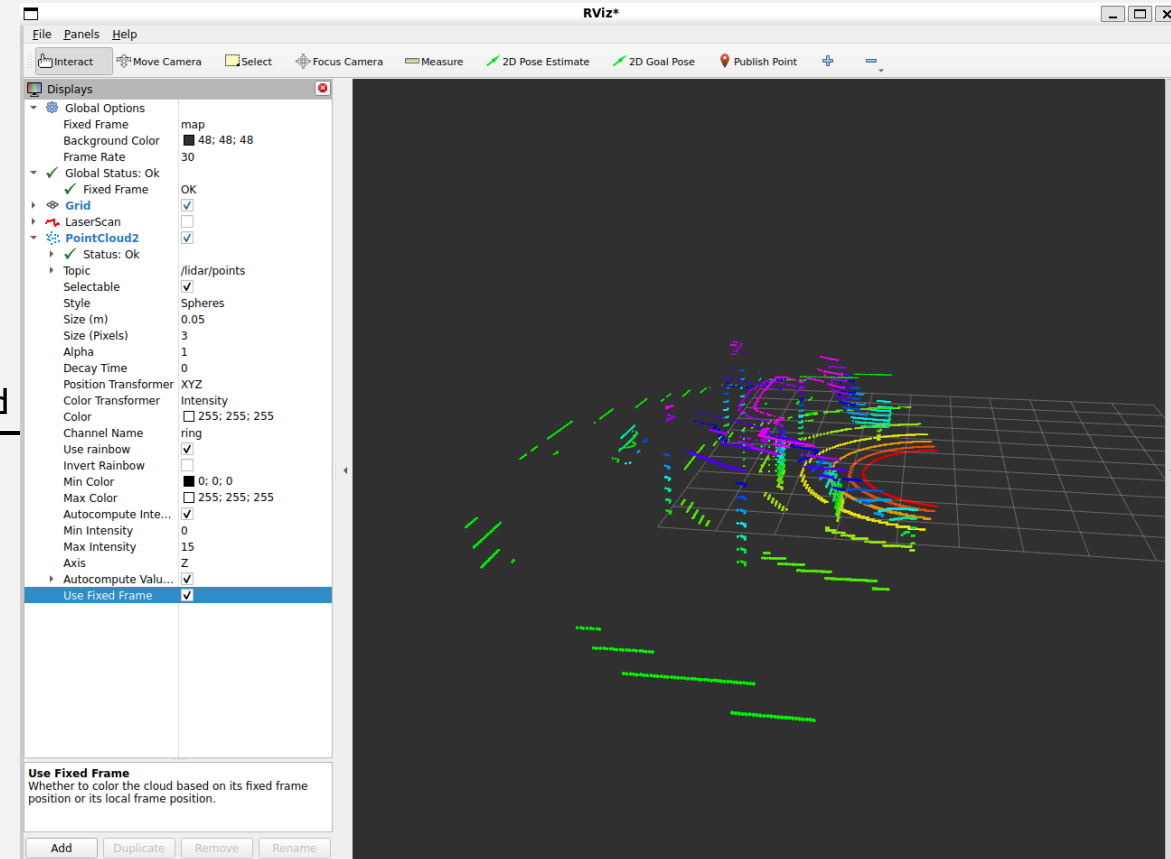
Point cloud (obtida a partir de LIDAR)

- Exemplo: simulado no Gazebo



gz.msgs.
PointCloudPacked

sensor_msgs/msg/
PointCloud2



Para rodar esse exemplo:
`ros2 launch ros_gz_sim_demos gpu_lidar_bridge.launch.py`